CHAPTER 22

Metaprogramming

The Jacquard loom, invented more than 200 years ago, was the first device controlled using punched cards—rows of holes in each card were used to control the pattern woven into the cloth. But imagine if instead of churning out fabric, the loom could punch more cards, and those cards could be fed back into the mechanism. The machine could be used to create new programming that it could then execute. And that would be metaprogramming—writing code that writes code.

Programming is all about building layers of abstractions. As you solve problems, you're building bridges from the unrelenting and mechanical world of silicon to the more ambiguous and fluid world we inhabit. Some programming languages—such as C—are close to the machine. The distance from C code to the application domain can be large. Other languages—Ruby, perhaps—provide higher-level abstractions and hence let you start coding closer to the target domain. For this reason, most people consider a higher-level language to be a better starting place for application development (although they'll argue about the choice of language).

But when you metaprogram, you are no longer limited to the set of abstractions built in to your programming language. Instead, you create new abstractions that are integrated into the host language. In effect, you're creating a new, domain-specific programming language—one that lets you express the concepts you need to solve your particular problem. Metaprogramming can be an excellent way to manage complex problems where the structure of the underlying data drives the structure of the code.

Ruby makes metaprogramming easy. As a result, Ruby programmers will often use metaprogramming techniques to simplify their code. This chapter shows how they do it. It isn't intended to be an exhaustive survey of metaprogramming techniques. Instead, we'll look at the underlying Ruby principles that make metaprogramming possible. From there you'll be able to invent your own metaprogramming idioms.

Understanding Objects and Classes

Classes and objects are central to Ruby, but at first sight they can be confusing. It seems like there are a lot of concepts: classes, objects, class objects, instance methods, class methods, singleton classes, and virtual classes. In reality, however, all these Ruby constructs are part of the same underlying class and object structure.